AURORA

Post-procesado QKD (BB84) en un SoC Zynq UltraScale+:
codiseño hardware/software, implementación y resultados en placa

Informe técnico

Noviembre de 2026

Resumen

Se describe la implementación del post-procesado de una distribución cuántica de claves BB84 en una placa ZCU104 (XCZU7EV). La parte con más volumen de datos (sincronización, *sifting*, *hash* de Toeplitz) se ejecuta en la lógica programable (PL) como *kernels* HLS en *streaming*; la corrección de errores, la autenticación Wegman–Carter del canal clásico y el control del protocolo se ejecutan en el procesador (PS). Un generador de números aleatorios verdaderos (TRNG) de osciladores en anillo en la PL proporciona las semillas públicas, con tests de salud SP 800-90B en hardware. Sobre 101 capturas reales (QBER 1,72 %), la placa reproduce bit a bit los resultados del modelo de referencia y entrega **28 377 bits de clave neta** a partir de 40 755 bits cribados. El TRNG tiene una entropía mínima medida de 0,785 bits/muestra (NIST `ea_non_iid`) y supera el test de reinicios de SP 800-90B. Se incluye, como alternativa a Cascade, una reconciliación LDPC de un solo sentido.

Índice

1. Plataforma y flujo de diseño **1**

2. Codiseño hardware/software **2**

3. Cadena de post-procesado **3**

4. Implementación en la PL **3**
4.1. Kernels HLS . . . 3
4.2. TRNG . . . 3
4.3. Implementación (Vivado) . . . 4

5. Resultados en placa **4**
5.1. Cadena completa (bare-metal, ZCU104) . . . 4
5.2. Entropía del TRNG . . . 4

6. Reconciliación: Cascade frente a LDPC **6**
6.1. Decodificador LDPC en la PL . . . 6

7. Verificación **7**

8. Limitaciones y trabajo futuro **7**

1. Plataforma y flujo de diseño

- **Placa**: Xilinx ZCU104, SoC XCZU7EV-FFVC1156-2-E (4×A53 a 1,2 GHz + PL UltraScale+).

- **Herramientas**: Vitis HLS, Vivado y Vitis 2022.2; aplicación *bare-metal* (standalone) en el A53#0.
- **Reloj de la PL**: `pl_clk0` = 100 MHz (objetivo HLS 10 ns).
- **Interfaces**: control por `M_AXI_HPM0_FPD` (AXI4-Lite). Los *kernels* acceden a la DDR por `S_AXI_HP0_FPD` a través de una única SmartConnect; el puerto no es coherente, así que el PS hace *flush* e *invalidate* de la caché.
- **Flujo reproducible** (`Makefile`): `make hls-all` → `make bitstream` → `make baremetal` → `make board-run`; los mismos fuentes C++ se compilan en el PC (tests y modelo de referencia), en HLS (*kernels*) y para el A53.

Tabla 1: Mapa de direcciones (AXI4-Lite).

Periférico	Base	Implementación
TRNG	`0xA000_0000`	RTL (Verilog)
`aurora_sync`	`0xA001_0000`	HLS
`aurora_sift`	`0xA002_0000`	HLS
`aurora_toeplitz`	`0xA003_0000`	HLS

2. Codiseño hardware/software

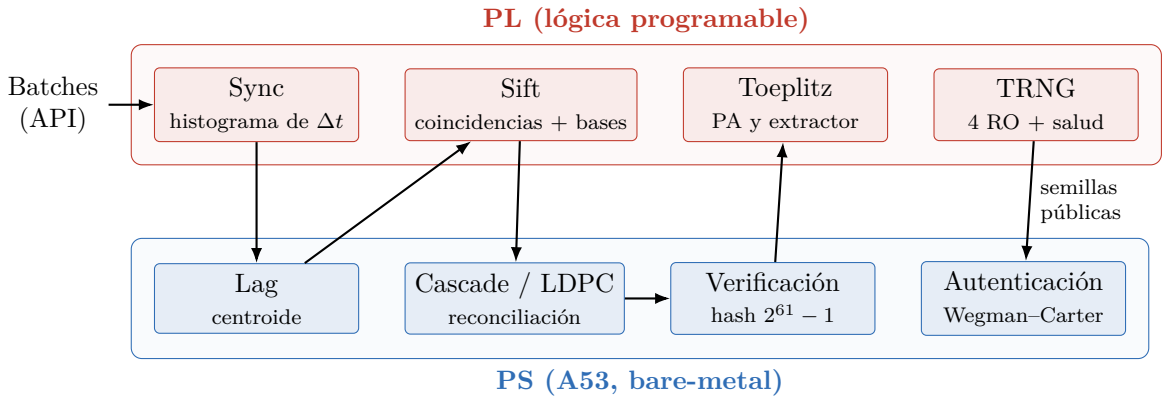


Figura 1: Reparto de la cadena de post-procesado entre la PL y el PS. Las flechas entre bloques indican el orden de la cadena; todos los datos pasan por la DDR.

¿Cascade en la PL? No compensa por ahora. Cascade necesita una ida y vuelta por el canal clásico por cada paso de su búsqueda binaria (cientos por bloque); cuando Alice y Bob estén en equipos distintos, la latencia de red dominará sobre el cálculo de paridades. Además, el A53 va sobrado: el sistema genera ∼16 kbit/s de clave cribada y el post-procesado completo en el PS procesa del orden de 10^5 bits/s. Si en el futuro la tasa sube a Mbit/s o la latencia del enlace es alta, la opción natural en la PL es un decodificador LDPC (un solo mensaje por trama, muy paralelizable); el código LDPC de la sección 6 está preparado con un decodificador *min-sum* normalizado, que es el adecuado para hardware.

Tabla 2: Reparto de cada etapa y justificación.

Etapa	Dónde	Motivo
TRNG (osciladores, tests RCT/APT, FIFO)	**PL**	Solo puede ser hardware; tests de salud continuos sin carga para la CPU.
Sync: histograma de diferencias de tiempo	**PL**	∼52 000 marcas por batch; ventana deslizante a 1 marca/ciclo.
Sync: centroide y elección del lag	**PS**	Pocas operaciones sobre 1024 bins.
Sifting	**PL**	Mezcla de dos flujos ordenados a 1 elemento/ciclo.
Cascade (o LDPC)	**PS**	Interactivo: dominado por los mensajes con el otro extremo, no por el cálculo. Lógica de control con muchas ramas.
Hash de verificación	**PS**	Un polinomio sobre ∼40 kbit: despreciable.
Amplificación de privacidad (Toeplitz)	**PL**	La operación más costosa: $O(n\,m)$ bits.
Extractor del TRNG (Toeplitz)	**PL**	Mismo *kernel* que la amplificación de privacidad.
Autenticación, reserva de clave, protocolo	**PS**	Control, red y estado.

3. Cadena de post-procesado

1. **Sincronización** por batch (25 ms): histograma de $\Delta t = t_A - t_B$ con paso grueso (1,024 ns) y fino (8 ps); lag por centroide con el fondo restado. Las capturas reales tienen un desfase distinto por batch (∼77 ps rms) y sin deriva dentro del batch: si la sincronización falla, el batch se descarta.

2. **Sifting**: emparejamiento voraz de detecciones dentro de ±350 ps y comparación de bases.

3. **Reconciliación**: Cascade (4 pases, $k_1 = 0{,}73/Q$, efecto cascada, límite de trabajo) o LDPC (sección 6).

4. **Verificación**: hash polinómico 2-universal módulo $2^{61} - 1$ con semilla nueva del TRNG (64 bits).

5. **Amplificación de privacidad**: Toeplitz con semilla completa de $n + m - 1$ bits por bloque; $m = n\,(1 - h(Q)) - \text{fuga}_{EC} - 64 - 2\log_2(1/\varepsilon)$ (fórmula asintótica).

6. **Autenticación**: Wegman–Carter, PolyHash sobre GF(2^{128}) cifrado con un OTP de 128 bits; cada fase se autentica en ambos sentidos (1152 bits de clave por ronda, repuestos con clave QKD). La clave se entrega solo si todas las etiquetas verifican.

4. Implementación en la PL

4.1. Kernels HLS

La primera versión de `sync` y `sift` no cerraba timing (17,2 y 17,6 ns frente a 10 ns) porque cada lectura `m_axi` dependía de la iteración anterior. Se reescribieron como `DATAFLOW` en *streaming*: lectores en ráfaga → núcleo con el estado en registros → escritor. En `sync`, el núcleo es una ventana deslizante de 32 marcas (máximo observado: 7) con reenvío en el histograma; si la ventana se desborda, el *kernel* lo indica (`complete = 0`) y el PS rechaza el lag.

4.2. TRNG

Cuatro osciladores en anillo de 3, 5, 7 y 9 etapas (LUT con `DONT_TOUCH`) combinados con XOR, sincronizador de dos biestables y decimación (1 muestra cada 64 ciclos: 1,56 Mmuestras/s). Tests de salud SP 800-90B en hardware (RCT, umbral 41; APT, ventana 1024, umbral 793; para $H_{\text{mín}} = 0{,}5$

Tabla 3: Síntesis HLS (estimación; objetivo 10 ns; todos los bucles internos con II = 1). Los recursos tras la implementación son menores (tabla 5).

Kernel	Estimado (ns)	LUT	FF	BRAM18	URAM	DSP
sync	7,300	13.258	7.462	14	0	0
sift	7,300	17.586	9.244	25	0	0
toeplitz	7,300	8.392	3.940	8	2	0
ldpc_dec	7,300	7.670	4.805	89	24	0

y $\alpha = 2^{-20}$), test de arranque de 1024 muestras, inyección de fallos para el autodiagnóstico y modo *bypass* solo para evaluación. El extractor (Toeplitz con semilla pública fija, bloques de 65 536 bits) se ejecuta en el *kernel* Toeplitz.

4.3. Implementación (Vivado)

- Timing: **WNS = +3,241 ns**, WHS = +0,009 ns, sin *endpoints* que fallen.
- DRC: 4 avisos LUTLP-2 (bucles combinacionales permitidos), uno por oscilador en anillo, como se espera.
- Potencia estimada: total 3,757 W (dinámica 3,060 W, estática 0,697 W), PS incluido.
- Kernel LDPC: por iteración, $\sim(d{+}7)$ ciclos por fila en la comprobación del síndrome y $\sim(2d{+}13)$ en la actualización (d: grado de la fila). Con el código de tasa 0,80 ($m = 3277$, $d \approx 25$) son ~3,1 ms por iteración a 100 MHz.

Tabla 4: Recursos del diseño completo.

Recurso	Usado	Disp.	%
LUT	30.140	230.400	13,08 %
Registros (FF)	42.764	460.800	9,28 %
BRAM (36 Kb)	64	312	20,51 %
URAM	26	96	27,08 %
DSP	0	1.728	0,00 %

Tabla 5: Recursos por módulo (implementado).

Módulo	LUT	FF	BRAM36/18	URAM	DSP
Kernel sync (HLS)	5.307	7.208	5/1	0	0
Kernel sift (HLS)	5.693	9.068	8/0	0	0
Kernel Toeplitz (HLS)	3.281	3.970	7/0	2	0
Kernel LDPC (HLS)	3.227	4.046	41/5	24	0
TRNG (RTL)	496	336	0/0	0	0
Interconexion AXI	486	568	0/0	0	0

5. Resultados en placa

5.1. Cadena completa (bare-metal, ZCU104)

Datos: 101 capturas reales de 25 ms, 79 532 coincidencias, 40 755 bits cribados, QBER 1,718 %. Sync + sift ocupan ~7,8 ms por batch de 25 ms (31 % del tiempo real), con margen para procesar en línea. El cuello de botella del post-procesado es la generación de semillas; se puede sacar del camino crítico generándolas en segundo plano mientras llegan los batches.

5.2. Entropía del TRNG

- La salida en bruto tiene un sesgo del 2 % ($P(0) = 0{,}521$) y falla los tests SP 800-22, como es de esperar; `serial_2`, que cancela el sesgo de primer orden, pasa (no hay correlación apreciable).
- La salida **extraída** (522 240 bits) pasa los 9 tests SP 800-22 (p entre 0,28 y 0,92).
- Se mantiene $H_{\text{mín}} = 0{,}5$ en el diseño: margen de 1,57 veces sobre el peor estimador.

Tabla 6: Prueba en placa: resultado **TODO OK (0 fallos)**.

Prueba	Resultado
TRNG en bruto	1 Mbit en 671 ms (1,56 Mmuestras/s), sin pérdidas ni alarmas
TRNG autodiagnóstico	Los 3 fallos inyectados disparan alarma; arranque correcto
TRNG extraído	100 000 bits en 615 ms (162 kbit/s), equilibrados ($z = 0{,}31$)
Sync + sift (3 batches)	Lag, coincidencias y claves **idénticos bit a bit** al PC; 5,1 ms + 2,7 ms por batch de 25 ms (incluye el trabajo del ARM)
Toeplitz	Idéntico a la definición (2000 → 1300 bits en 0,34 ms)
Cadena autenticada	40 755 cribados → 29 529 tras PA → **28 377 bits netos**; claves iguales; 709 ms (de ellos, ∼430 ms generando 70 409 bits de semillas con el TRNG, estimado)

Tabla 7: Evaluación SP 800-90B (`ea_non_iid`, 1 Mbit en bruto, temperatura ambiente).

Estimador	H (bit/muestra)
Most Common Value	0,937
Collision	0,910
Markov	0,940
Compression	**0,785**
t-Tuple / LRS	0,912 / 0,970
MultiMCW / Lag / MultiMMC / LZ78Y	0,944 / 0,992 / 0,938 / 0,938
H_{original}	**0,785**

Tabla 8: Test de reinicios SP 800-90B §3.1.4 (1000 reinicios × 1000 muestras, `ea_restart`, $H_I = 0{,}785$). Cordura: frecuencia máxima del valor más común en filas y columnas frente al umbral.

Modo	Descarte	Cordura	H_r	H_c	mín(H_r, H_c, H_I)
Parada/arranque	0	564 / 648	0,779	0,803	**0,779**
Parada/arranque	1024	567 / 648	0,781	0,778	**0,778**
Corte de alimentación	0	564 / 648	0,810	0,795	**0,785**

Tabla 9: Entropía mínima (H_{original}, `ea_non_iid`) frente a la temperatura del chip.

Captura	T PL (°C)	Compression	H_{original}
Ambiente (sin sensor)	–	0,785	0,785
Frío (recién encendida)	28,8	0,764	0,764
Normal (ventilador, >15 min)	29,8–30,1	0,786	0,786
Caliente	*pendiente*	*pendiente*	*pendiente*

No hay transitorio de arranque: desde la primera muestra tras habilitar los osciladores, el valor más común entre reinicios aparece con frecuencia 0,50–0,56, también tras un corte de alimentación real (1000 ciclos con un relé USB, ∼11 h). La entropía tras reinicio es igual o mayor que la de funcionamiento continuo: el TRNG queda validado frente a reinicios. Pendiente: capturas en frío y en caliente (la aplicación registra la temperatura del chip en cada captura).

6. Reconciliación: Cascade frente a LDPC

La reconciliación LDPC es de un solo sentido por trama: Alice envía el síndrome y Bob responde con éxito y número de errores. Se usa una familia de códigos IRA ($n = 16\,384$, tasas 0,50–0,95) construidos de forma determinista (sin ciclos de longitud 4 hasta $R = 0{,}90$), adaptación de tasa con perforado *untainted* y acortado, QBER adaptativo trama a trama, y decodificación BP por capas (*sum-product* o *min-sum* normalizado). Las tramas que fallan se descartan.

Tabla 10: Tasa de error de trama (FER) del LDPC con $Q = 1{,}72\,\%$ (simulación, sum-product, 100 iteraciones).

f objetivo	1,20	1,25	1,30	1,35	1,40	1,45	1,50
FER	13 %	8 %	8 %	2,5 %	0/124	0/120	0/120

Tabla 11: Cadena autenticada sobre los 40 755 bits cribados reales (PC).

Método	QBER estimado	f	Fuga	Clave neta	Mensajes	Tiempo (PC)
Cascade	2,00 %	1,172	5 985	28 381	∼6 000 consultas	0,05 s
Cascade	1,72 %	1,154	5 891	28 475	∼6 000 consultas	0,05 s
LDPC	2,00 %	1,480	7 556	26 810	3 + 3	0,96 s
LDPC	1,72 %	1,396	7 125	27 241	3 + 3	0,68 s

6.1. Decodificador LDPC en la PL

El *kernel* `aurora_ldpc_dec` implementa la decodificación de Bob con *min-sum* normalizado por capas en coma fija: LLR de entrada y mensajes de 8 bits, posteriores de 16 bits y normalización $\alpha = 0{,}6875 = \frac{1}{2} + \frac{1}{4} - \frac{1}{16}$ (solo sumas y desplazamientos). El código (matriz H en CSR) se guarda en memoria interna (URAM/BRAM) y solo se recarga al cambiar de código. El mismo fuente C++ es el modelo de software, de modo que la comparación PL/PS es exacta bit a bit. Las etapas de planificación, perforado, síndrome de Alice y mensajes siguen en el PS.

Tabla 12: FER del decodificador en coma fija ($Q = 1{,}72\,\%$, 300 tramas por punto) frente a sum-product en flotante.

	Min-sum coma fija (PL)			Sum-product (PS)
f objetivo	1,50	1,55	1,60	1,40
FER	1,7 %	0,7 %	1,0 %	0/151

El *min-sum* en coma fija necesita más margen ($f = 1{,}50$) y tiene un suelo de error de ∼1 %: con los datos reales entrega ∼25 800 bits netos frente a 27 241 con sum-product. A cambio, es la variante que se puede llevar a hardware.

La PL y el A53 obtienen **resultados idénticos bit a bit** (mismas claves, fuga, tramas fallidas e iteraciones), y la PL es **1,3 veces más rápida** con el mismo algoritmo, en línea con la estimación

Tabla 13: Decodificación LDPC de los 40 755 bits reales en la placa: PL frente a PS (`make board-run`, sección 6).

Decodificador	Tiempo	Resultado	Clave neta
PL, min-sum coma fija	**384 ms**	4 tramas, 1 fallida	25 845
PS (A53), min-sum coma fija	502 ms	4 tramas, 1 fallida	25 841
PS (A53), sum-product flotante	617 ms	3 tramas, 0 fallidas	–

por síntesis (∼3,1 ms por iteración frente a ∼3,9 ms en el A53). La clave neta difiere en unos pocos bits entre ejecuciones porque las semillas de la amplificación de privacidad salen del TRNG.
Medido en la placa: el A53 reconcilia los 40 755 bits en 0,5–0,6 s, mientras que se tardan ∼2,5 s en generarlos, así que el PS ya sigue el ritmo con un margen de ∼4 veces (la estimación previa de 2–3 s era pesimista). La construcción de los códigos tarda 3,2 s, una sola vez al arrancar. En la ejecución en coma fija, la trama que falla agota las 100 iteraciones y se lleva buena parte del tiempo. El *kernel* procesa una arista por ciclo (100 M aristas/s): con ∼83 000 aristas, tres pasadas por iteración y ∼10 iteraciones, unos 30 ms por trama, del mismo orden que el A53. Con las tasas actuales la PL no es necesaria para el LDPC; lo sería si la tasa de clave creciera. Para una ganancia grande habría que procesar varias filas en paralelo, lo que exige códigos cuasi-cíclicos (QC-LDPC): es el siguiente paso si la medida en placa lo justifica.
Con este tamaño de trama, el LDPC entrega un 4 % menos de clave que Cascade, pero con 6 mensajes en lugar de miles de consultas: es la opción a elegir cuando la latencia de red domine o cuando se lleve la decodificación a la PL. La eficiencia mejoraría con tramas mayores ($n \geq 10^5$) y distribuciones de grados optimizadas por evolución de densidad.

7. Verificación

- **Tests en PC** (`make test-all`): 73 tests C++ (*kernels*, autenticación, LDPC, TRNG, driver), 34 tests Python (incluida la validación sobre los 101 batches reales contra un modelo independiente), tests que documentan los 9 fallos de la implementación anterior, y 6 simulaciones RTL del TRNG (Icarus).
- **C-simulation HLS** de los tres *kernels* con el mismo *testbench*.
- **Plataforma simulada**: la aplicación *bare-metal* completa se ejecuta en el PC con los periféricos emulados a nivel de registro.
- **Placa**: resultados idénticos bit a bit a los del PC; evaluación de entropía con las herramientas de NIST.
- **Seguridad del canal**: se comprueba que cualquier alteración de paridades, síndromes, mensajes o etiquetas, o la repetición de una etiqueta, aborta la ronda sin entregar clave.

8. Limitaciones y trabajo futuro

- **Seguridad**: la longitud de la amplificación de privacidad usa la fórmula asintótica de BB84, sin correcciones de tamaño finito; debe validarse con el equipo de UPM.
- **Integración con la API del receptor** y paso a Linux (PetaLinux) con *buffers* contiguos.
- **Alice y Bob en procesos separados**, con los anuncios del *sifting* dentro de la transcripción autenticada.

- **Rendimiento en el PS**: medir el reparto de tiempos por etapa; generar las semillas en segundo plano; representar las claves empaquetadas (64 bits por palabra) y usar NEON (`PMULL` para GF(2^{128})).
- **QRNG del receptor**: si la API lo expone, combinarlo con el TRNG (XOR).
- **LDPC**: tramas mayores, distribuciones optimizadas; en la PL, códigos QC-LDPC para decodificar varias filas en paralelo.